CHAPTER 1 · SUB-CHAPTER 1.9

Java 8 Features

Lambda Expressions · Functional Interfaces · Stream API · Collectors
Optional · Default & Static Interface Methods · Date/Time API · Method References

~20 Pages	52+ Q&A	Adv. Questions	Interview Ready

■ Purple [ADV] questions indicate advanced / senior-level depth

Section 1 — Lambda Expressions & Method References

1.1 Lambda Syntax

```
// Lambda forms: (parameters) -> expression OR (parameters) -> { block; }
Runnable r1 = () -> System.out.println("hello"); // no param, expression
Comparator<String> c1 = (a, b) -> a.compareTo(b); // two params, expression
Comparator<String> c2 = (String a, String b) -> { // explicit types, block
int cmp = a.compareTo(b);
return cmp;
};
// Effectively final: captured variables must not be reassigned after capture
String prefix = "Hello";
// prefix = "Hi"; // would make it NOT effectively final — compile error
Supplier<String> greeter = () -> prefix + " World"; // prefix is effectively final
// Lambda vs Anonymous Inner Class differences:
// 1. 'this' inside lambda = enclosing class; inside anon class = anon class itself
// 2. Lambda has no separate .class file (uses invokedynamic)
// 3. Lambda can capture 'effectively final' locals; anon class needs 'final'
// 4. Lambda is more concise and can be inlined by JIT more easily
// Lambda as Comparator
List<String> names = new ArrayList<>(List.of("Charlie","Alice","Bob"));
names.sort((a, b) -> a.length() - b.length()); // sort by length
names.sort(Comparator.comparing(String::length)); // preferred — avoids int subtraction overflow
```

1.2 Four Types of Method References

Type	Syntax	Lambda Equivalent	Example
Static method	Type::staticMethod	(args) -> Type.staticMethod(args)	Integer::parseInt
Instance (unbound)	Type::instanceMethod	(inst, args) -> inst.instanceMethod(args)	String::toUpperCase
Instance (bound)	obj::instanceMethod	(args) -> obj.instanceMethod(args)	System.out::println
Constructor	Type::new	(args) -> new Type(args)	ArrayList::new

```
// All four types in practice
Function<String,Integer> parse = Integer::parseInt; // static
Function<String,String> upper = String::toUpperCase; // unbound instance
Consumer<String> printer = System.out::println; // bound instance
Supplier<List<String>> listMaker= ArrayList::new; // constructor ref
// Unbound vs Bound:
String greeting = "hello";
Supplier<String> bound = greeting::toUpperCase; // bound to 'greeting'
Function<String,String> unbound = String::toUpperCase; // applies to any String
// Practical examples
List<String> words = List.of("alpha","beta","gamma");
words.stream().map(String::toUpperCase).forEach(System.out::println);
words.stream().map(String::length).sorted().forEach(System.out::println);
List<Thread> threads = List.of(new Thread(() -> {}));
threads.stream().map(Thread::getName).forEach(System.out::println);
```

Section 2 — Functional Interfaces (java.util.function)

Interface	SAM Signature	Key Default Methods	Typical Use
Predicate	boolean test(T t)	and(), or(), negate(), not() (Java 11)	Filtering, condition testing
Function	R apply(T t)	andThen(fn), compose(fn), identity()	Transforming values
BiFunction	R apply(T t, U u)	andThen(fn)	Combining two inputs
Consumer	void accept(T t)	andThen(consumer)	Side effects (print, save)
BiConsumer	void accept(T t, U u)	andThen(consumer)	Two-arg side effects
Supplier	T get()	(none)	Lazy values, factories
UnaryOperator	T apply(T t)	(extends Function)	In-place transformation
BinaryOperator	T apply(T t1, T t2)	maxBy(), minBy()	Combining same-type values
Comparator	int compare(T o1, T o2)	comparing(), reversed(), thenComparing()	Ordering/sorting

```
// Predicate
Predicate<String> nonEmpty = s -> !s.isEmpty();
Predicate<String> longWord = s -> s.length() > 5;
Predicate<String> nonEmptyLong = nonEmpty.and(longWord); // AND
Predicate<String> emptyOrLong = nonEmpty.negate().or(longWord); // OR
Predicate<String> isNull = Objects::isNull;
Predicate<String> notNull = Predicate.not(Objects::isNull); // Java 11
// Function composition
Function<String,Integer> strlen = String::length;
Function<Integer,Boolean> isEven = n -> n % 2 == 0;
Function<String,Boolean> hasEvenLen = strlen.andThen(isEven); // strlen THEN isEven
Function<String,Boolean> composed = isEven.compose(strlen); // strlen THEN isEven (same)
// andThen(g): f.andThen(g) = x -> g(f(x))
// compose(f): g.compose(f) = x -> g(f(x)) — equivalent when g=isEven, f=strlen
// Comparator chaining
Comparator<Person> byAge = Comparator.comparing(Person::getAge);
Comparator<Person> byName = Comparator.comparing(Person::getName);
Comparator<Person> combined = byAge.reversed().thenComparing(byName);
Comparator<String> nullSafe = Comparator.nullsFirst(Comparator.naturalOrder());
// Primitive specialisations (avoid boxing)
IntPredicate isPositive = n -> n > 0;
IntFunction<String> intToStr= Integer::toString;
ToIntFunction<String> toLen = String::length;
IntUnaryOperator doubler = n -> n * 2;
IntBinaryOperator add = Integer::sum;
```

Section 3 — Stream API

3.1 Pipeline Structure & Lazy Evaluation

> Streams are LAZY: intermediate operations build a pipeline but execute nothing. Only a terminal operation triggers execution. Short-circuit terminals (findFirst, anyMatch) stop processing as soon as the answer is known.

```
// Stream pipeline: source -> intermediate ops (lazy) -> terminal op (eager)
List<String> names = List.of("Alice","Bob","Charlie","Anna","Alex","Dave");
// Complex pipeline — executes as ONE pass (fusion)
List<String> result = names.stream() // source
.filter(n -> n.startsWith("A")) // intermediate (lazy)
.map(String::toUpperCase) // intermediate (lazy)
```

```
.sorted() // stateful intermediate (buffers)
.limit(3) // short-circuit intermediate
.collect(Collectors.toList()); // terminal — triggers execution
// Stream sources
Stream<String> fromList = names.stream();
Stream<Integer> fromArray = Arrays.stream(new Integer[]{1,2,3});
Stream<String> ofValues = Stream.of("a","b","c");
Stream<Double> generated = Stream.generate(Math::random).limit(10);
Stream<Integer> iterated = Stream.iterate(0, n -> n + 2).limit(5); // 0,2,4,6,8
Stream<String> fileLines = Files.lines(Path.of("data.txt")); // must close!
IntStream range = IntStream.range(0, 10); // 0..9
IntStream rangeClosed= IntStream.rangeClosed(1, 10); // 1..10
// Flat map: one-to-many transformation
List<List<Integer>> nested = List.of(List.of(1,2), List.of(3,4), List.of(5));
List<Integer> flat = nested.stream()
.flatMap(Collection::stream) // Stream<List<Integer>> -> Stream<Integer>
.collect(Collectors.toList()); // [1,2,3,4,5]
// FlatMap for words in sentences
List<String> sentences = List.of("hello world", "java streams");
List<String> words = sentences.stream()
.flatMap(s -> Arrays.stream(s.split(" ")))
.collect(Collectors.toList());
```

3.2 All Terminal Operations

Terminal Op	Returns	Short-Circuit ?	Notes
collect(Collector)	R	No	Most flexible; Collectors.toList/Map/groupingBy
forEach(Consumer)	void	No	Order not guaranteed on parallel
forEachOrdered(Consumer)	void	No	Preserves encounter order, even parallel
count()	long	No	Size of stream
reduce(identity, BinaryOp)	T	No	Fold: sum, product, concat
reduce(BinaryOp)	Optional	No	No identity — empty stream returns empty
min(Comparator)	Optional	No	Smallest element
max(Comparator)	Optional	No	Largest element
findFirst()	Optional	YES	First in encounter order
findAny()	Optional	YES	Any element; faster on parallel
anyMatch(Predicate)	boolean	YES	True if any matches; false on empty
allMatch(Predicate)	boolean	YES	True if all match; TRUE on empty
noneMatch(Predicate)	boolean	YES	True if none match; TRUE on empty
toArray()	Object[]	No	Or toArray(String[]::new)

3.3 reduce() and Numeric Streams

```
// reduce — three overloads
// 1. Identity + associative BinaryOperator
int sum1 = IntStream.rangeClosed(1,10).reduce(0, Integer::sum); // 55
// 2. No identity — returns Optional (handles empty stream)
Optional<Integer> sum2 = Stream.of(1,2,3).reduce(Integer::sum); // Optional[6]
// 3. Identity + accumulator + combiner (for parallel streams)
int sum3 = Stream.of(1,2,3,4,5).reduce(0,
(subtotal, n) -> subtotal + n, // accumulator (sequential)
Integer::sum); // combiner (merges parallel results)
```

```
// Numeric streams — avoid boxing overhead
int[] arr = {1,2,3,4,5};
int total = IntStream.of(arr).sum(); // 15
double avg = IntStream.of(arr).average().orElse(0); // 3.0
IntSummaryStatistics stats = IntStream.of(arr).summaryStatistics();
// stats.getMin(), getMax(), getSum(), getAverage(), getCount()
// boxed() / asLongStream / asDoubleStream
Stream<Integer> boxed = IntStream.range(1,5).boxed(); // int -> Integer
LongStream asLong = IntStream.range(1,5).asLongStream();
DoubleStream asDouble= IntStream.range(1,5).asDoubleStream();
// mapToObj, mapToInt, mapToLong, mapToDouble
Stream<String> strs = IntStream.rangeClosed('A','Z')
.mapToObj(c -> String.valueOf((char)c));
```

Section 4 — Collectors

```
import java.util.stream.Collectors;
List<Employee> emps = getEmployees(); // name, dept, salary (int)
// Basic collection
List<String> names = emps.stream().map(Employee::getName).collect(Collectors.toList());
Set<String> depts = emps.stream().map(Employee::getDept).collect(Collectors.toSet());
List<String> immutable = emps.stream().map(Employee::getName)
.collect(Collectors.toUnmodifiableList()); // Java 10+
// toMap — key extractor, value extractor
Map<String,Integer> nameSalary = emps.stream()
.collect(Collectors.toMap(Employee::getName, Employee::getSalary));
// Duplicate key: provide merge function
Map<String,Integer> nameMaxSalary = emps.stream()
.collect(Collectors.toMap(Employee::getName, Employee::getSalary,
Integer::max)); // keep higher on duplicate key
// groupingBy — group into Map<K, List<V>>
Map<String,List<Employee>> byDept = emps.stream()
.collect(Collectors.groupingBy(Employee::getDept));
// groupingBy with downstream collector
Map<String,Long> countByDept = emps.stream()
.collect(Collectors.groupingBy(Employee::getDept, Collectors.counting()));
Map<String,Double> avgSalByDept = emps.stream()
.collect(Collectors.groupingBy(Employee::getDept,
Collectors.averagingInt(Employee::getSalary)));
Map<String,Integer> sumSalByDept = emps.stream()
.collect(Collectors.groupingBy(Employee::getDept,
Collectors.summingInt(Employee::getSalary)));
Map<String,Optional<Employee>> topByDept = emps.stream()
.collect(Collectors.groupingBy(Employee::getDept,
Collectors.maxBy(Comparator.comparingInt(Employee::getSalary))));
// partitioningBy — splits into Map<Boolean, List<T>>
Map<Boolean,List<Employee>> highLow = emps.stream()
.collect(Collectors.partitioningBy(e -> e.getSalary() > 80_000));
List<Employee> highPaid = highLow.get(true);
// joining
String csv = emps.stream().map(Employee::getName).collect(Collectors.joining(", "));
String csvBrkt = emps.stream().map(Employee::getName).collect(Collectors.joining(", ","["]"));
// mapping + downstream
Map<String,Set<String>> namesByDept = emps.stream()
.collect(Collectors.groupingBy(Employee::getDept,
Collectors.mapping(Employee::getName, Collectors.toSet())));
// collectingAndThen — apply finisher after downstream
Map<String,List<Employee>> unmodByDept = emps.stream()
.collect(Collectors.groupingBy(Employee::getDept,
Collectors.collectingAndThen(Collectors.toList(),
Collections::unmodifiableList)));
```

Section 5 — Optional

5.1 Creation & Consumption

```
// Creating Optional
Optional<String> full = Optional.of("value"); // must be non-null (NPE if null)
Optional<String> maybe = Optional.ofNullable(getValue()); // null -> empty
```

```
Optional<String> empty = Optional.empty();
// Consuming — SAFE patterns
String value1 = maybe.orElse("default"); // always evaluates "default"
String value2 = maybe.orElseGet(() -> computeDefault()); // lazy — only if empty
String value3 = maybe.orElseThrow(() -> new NotFoundException("id"));
// Transforming
Optional<Integer> length = maybe.map(String::length); // Optional[5] or empty
Optional<String> upper = maybe.map(String::toUpperCase);
Optional<String> flatMapped= maybe.flatMap(this::findByName); // avoids Optional<Optional>
// Filtering
Optional<String> longStr = maybe.filter(s -> s.length() > 3);
// Java 9+
maybe.ifPresentOrElse(
s -> System.out.println("Found: " + s),
() -> System.out.println("Not found"));
Optional<String> fallback = maybe.or(() -> Optional.of("fallback")); // Java 9+
Stream<String> stream = maybe.stream(); // 0 or 1 element stream (Java 9+)
// UNSAFE — avoid!
// String bad = maybe.get(); // NoSuchElementException if empty — never use without isPresent check
```

5.2 orElse vs orElseGet — Critical Difference

orElse(value): the value argument is ALWAYS evaluated, even when Optional is present. orElseGet(supplier): the supplier is called ONLY when Optional is empty (lazy). Use orElseGet for any non-trivial default to avoid wasted computation.

```
// orElse — ALWAYS executes the argument
Optional<String> opt = Optional.of("present");
String v1 = opt.orElse(expensiveQuery()); // expensiveQuery() RUNS even though opt has value!
// orElseGet — only executes supplier when empty
String v2 = opt.orElseGet(() -> expensiveQuery()); // expensiveQuery() SKIPPED — opt has value
// Rule: use orElse for cheap constants/literals; orElseGet for any computation
String safe1 = opt.orElse("N/A"); // fine — trivial constant
String safe2 = opt.orElseGet(this::loadDefault); // fine — lazy computation
// Optional chaining — pipeline style (no nested null checks)
// Old style:
User user = findUser(id);
String city = null;
if (user != null) {
Address addr = user.getAddress();
if (addr != null) city = addr.getCity();
}
// Optional style:
String city2 = findOptionalUser(id)
.map(User::getAddress)
.map(Address::getCity)
.orElse("Unknown");
```

5.3 Optional Anti-Patterns

Anti-Pattern	Why Bad	Correct Approach
Optional as field	Serialisation issues; adds memory overhead per field	Use @Nullable, null check in getter
Optional as method param	Callers can pass empty to avoid null check; ugly API	Use overloading or @Nullable parameter
Optional in collections	Optional inside List/Map defeats purpose	Use non-null values; filter out nulls at source

optional.get() without check	NoSuchElementException if empty	Use orElse/orElseGet/orElseThrow
if(opt.isPresent()) opt.get()	Verbose; misses Optional's purpose	Use map/filter/ifPresent/orElse chains
Optional.of(possiblyNull)	NPE if null	Optional.ofNullable(possiblyNull)

Section 6 — Default & Static Interface Methods

```
// Default method — provides implementation in interface
// Enables interface evolution without breaking existing implementations
interface Collection<E> {
// ... existing abstract methods ...
default void forEach(Consumer<? super E> action) { // added in Java 8
Objects.requireNonNull(action);
for (E e : this) { action.accept(e); }
}
default Stream<E> stream() { return StreamSupport.stream(spliterator(), false); }
default Spliterator<E> spliterator() { return Spliterators.spliteratorUnknownSize(iterator(), 0); }
}
// Diamond problem — resolved by implementing class
interface A { default void hello() { System.out.println("A"); } }
interface B extends A { default void hello() { System.out.println("B"); } }
interface C extends A { default void hello() { System.out.println("C"); } }
class D implements B, C {
@Override public void hello() {
B.super.hello(); // MUST override; choose explicitly which super to call
}
}
// Rules for default method conflict resolution:
// 1. Class always wins over interface default
// 2. More specific interface wins (B extends A — B's default wins over A's)
// 3. If still ambiguous — compiler error; class MUST override
// Static interface methods — utility methods on the interface itself
interface Validator<T> {
boolean validate(T value);
static <T> Validator<T> of(Predicate<T> predicate) { // factory method
return value -> predicate.test(value);
}
default Validator<T> and(Validator<T> other) {
return value -> this.validate(value) && other.validate(value);
}
}
Validator<String> nonEmpty = Validator.of(s -> !s.isEmpty());
Validator<String> maxLen10 = Validator.of(s -> s.length() <= 10);
Validator<String> combined = nonEmpty.and(maxLen10);
```

Section 7 — java.time Date/Time API

Class	Description	Has Timezone?	Immutable?
LocalDate	Date only: 2024-01-15	No	Yes
LocalTime	Time only: 10:30:45	No	Yes
LocalDateTime	Date + time (no zone)	No	Yes
ZonedDateTime	Date + time + zone ID	Yes (ZoneId)	Yes
OffsetDateTime	Date + time + UTC offset	Yes (ZoneOffset)	Yes
Instant	Nanosecond UTC timestamp (epoch)	UTC only	Yes
Duration	Amount of time (seconds/nanos)	No	Yes
Period	Date-based amount (years/months/days)	No	Yes
ZoneId	Time-zone (e.g. Europe/London)	Yes	Yes

All java.time classes are IMMUTABLE and THREAD-SAFE — unlike java.util.Date.

```
// LocalDate
LocalDate today = LocalDate.now();
LocalDate bday = LocalDate.of(1990, Month.JUNE, 15);
LocalDate parsed = LocalDate.parse("2024-01-15"); // ISO-8601 default
int age = Period.between(bday, today).getYears();

LocalDate nextWeek = today.plusWeeks(1);
LocalDate lastMonth = today.minusMonths(1);
LocalDate firstDay = today.withDayOfMonth(1);
boolean isLeap = today.isLeapYear();
DayOfWeek dow = today.getDayOfWeek(); // DayOfWeek.MONDAY etc.

// LocalDateTime & ZonedDateTime
LocalDateTime now = LocalDateTime.now();
ZonedDateTime zoned = ZonedDateTime.now(ZoneId.of("America/New_York"));
ZonedDateTime utc = zoned.withZoneSameInstant(ZoneId.of("UTC")); // convert zone

// Instant — machine-readable timestamp
Instant instant = Instant.now();
long epochMs = instant.toEpochMilli();
Instant future = instant.plus(Duration.ofHours(24));

// DateTimeFormatter — thread-safe (unlike SimpleDateFormat)
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
String formatted = LocalDateTime.now().format(fmt);
LocalDateTime parsed2 = LocalDateTime.parse("15/01/2024 10:30", fmt);

// Predefined formatters
DateTimeFormatter iso = DateTimeFormatter.ISO_LOCAL_DATE;
DateTimeFormatter http = DateTimeFormatter.RFC_1123_DATE_TIME;

// Legacy interop
Date legacyDate = Date.from(Instant.now());
Instant fromDate = legacyDate.toInstant();
LocalDate fromDate2 = legacyDate.toInstant().atZone(ZoneId.systemDefault()).toLocalDate();

// Duration and Period
Duration threeHours = Duration.ofHours(3);
Duration fromUntil = Duration.between(LocalTime.of(9,0), LocalTime.of(17,30));
Period twoMonths = Period.ofMonths(2);
Period diff = Period.between(LocalDate.of(2020,1,1), today);
```

Section 8 — Parallel Streams, Map Improvements & Other Java 8 APIs

8.1 Parallel Streams

```
// Parallel stream: splits work across ForkJoinPool.commonPool() threads
List<Integer> numbers = IntStream.rangeClosed(1, 1_000_000)
.boxed().collect(Collectors.toList());
long sum = numbers.parallelStream()
.filter(n -> n % 2 == 0) // parallel filter
.mapToLong(Integer::longValue) // parallel map
.sum(); // parallel reduce (associative — safe)
// Custom thread pool for parallel streams (avoid blocking commonPool)
ForkJoinPool customPool = new ForkJoinPool(4);
long result = customPool.submit(() ->
numbers.parallelStream().mapToLong(Integer::longValue).sum()
).get();
// WHEN to use parallel streams:
// GOOD: large data (>10K elements), CPU-bound, stateless operations, no ordering needed
// BAD: small data (overhead > benefit), IO-bound (ties up ForkJoinPool), stateful ops,
// operations with shared mutable state, ordered output required from parallel unordered data
// Stateful danger — NEVER do this:
List<Integer> results = new ArrayList<>(); // NOT thread-safe
numbers.parallelStream().filter(n -> n > 0).forEach(results::add); // DATA RACE!
// Fix: collect into thread-safe structure
List<Integer> safeResults = numbers.parallelStream()
.filter(n -> n > 0).collect(Collectors.toList()); // Collector handles concurrency
```

8.2 Map Improvements (Java 8)

```
Map<String,Integer> map = new HashMap<>();
// getOrDefault — single lookup (vs containsKey + get)
int val = map.getOrDefault("key", 0);
// putIfAbsent — atomic check-and-put
map.putIfAbsent("key", 42);
// computeIfAbsent — lazy value creation (Supplier)
map.computeIfAbsent("key", k -> k.length()); // only computes if absent
// computeIfPresent — update only if key exists
map.computeIfPresent("key", (k, v) -> v + 1);
// compute — always; null return removes the key
map.compute("count", (k, v) -> v == null ? 1 : v + 1); // increment or init
// merge — most powerful: insert/update/delete in one atomic call
map.merge("word", 1, Integer::sum); // frequency count
// if absent: put("word", 1)
// if present: put("word", existing + 1)
// if function returns null: removes the key
// replaceAll — update all values in-place
map.replaceAll((k, v) -> v.toUpperCase()); // Map<String,String>
// forEach — iterate entries
map.forEach((k, v) -> System.out.println(k + "=" + v));
```

8.3 Base64, StringJoiner & Files Streams

```
// Base64 encoding/decoding (Java 8)
byte[] data = "Hello, World!".getBytes();
String encoded = Base64.getEncoder().encodeToString(data);
byte[] decoded = Base64.getDecoder().decode(encoded);
```

```
// URL-safe encoder (replaces + with - and / with _)
String urlSafe = Base64.getUrlEncoder().encodeToString(data);
// MIME encoder (line breaks every 76 chars)
String mime = Base64.getMimeEncoder().encodeToString(data);
// StringJoiner (behind Collectors.joining)
StringJoiner sj = new StringJoiner(", ", "[", "]");
sj.add("Alice"); sj.add("Bob"); sj.add("Charlie");
System.out.println(sj); // [Alice, Bob, Charlie]
// Files stream-based operations (Java 8)
try (Stream<String> lines = Files.lines(Path.of("data.txt"))) {
long count = lines.filter(l -> !l.isBlank()).count();
} // stream auto-closed by TWR
try (Stream<Path> files = Files.walk(Path.of("/tmp"))) {
files.filter(Files::isRegularFile)
.filter(p -> p.toString().endsWith(".log"))
.forEach(System.out::println);
}
```

Interview Q&A; — 52 Questions

Purple [ADV] questions target senior/principal-level interviews.

Part 1 — Lambdas, Method References & Functional Interfaces

Q1. What is a lambda expression and what is required to use one?

A lambda is an anonymous function: (params) -> body. It can be used wherever a functional interface (interface with exactly one abstract method — SAM) is expected. The lambda provides the implementation of that SAM. The parameter types can be inferred or explicit. Captured local variables must be effectively final (not reassigned after capture).

Q2. What does 'effectively final' mean and why is it required for lambda captures?

A variable is effectively final if it is never reassigned after its initial assignment — even without the final keyword. Required because lambdas may outlive the stack frame where the variable was declared (e.g., lambda stored in a field, used asynchronously). If the captured variable could change, the lambda would observe stale or inconsistent values. Instance fields and array elements (even if technically mutable) can be accessed since the reference (not the value) is captured.

Q3. What are the 4 types of method references? Give an example of each.

1) Static: Integer::parseInt — (s) -> Integer.parseInt(s). 2) Unbound instance: String::toUpperCase — (s) -> s.toUpperCase(). 3) Bound instance: System.out::println — (x) -> System.out.println(x). 4) Constructor: ArrayList::new — () -> new ArrayList(). Unbound vs bound: unbound takes the instance as first arg (good for stream.map); bound is tied to a specific object.

Q4. What is a functional interface? Can it have default or static methods?

A functional interface has exactly one abstract method (SAM). It CAN have any number of default methods, static methods, and methods inherited from Object (equals, hashCode, toString). @FunctionalInterface annotation enforces this at compile time but is not required. Examples: Runnable, Callable, Comparator, Predicate, Function, Consumer, Supplier. Comparator has many default methods (reversed, thenComparing) but only one abstract method (compare).

Q5. What is the difference between Predicate.and() and &&?

Predicate.and(other) creates a new Predicate that tests both predicates compositely — it is a method returning a composed predicate object, usable in streams and chains. && is a Java operator for booleans directly. Predicate.and() short-circuits (if first is false, second is not tested). andThen/compose are for Function; and/or/negate are for Predicate. Java 11 added Predicate.not(predicate) as a static factory for negation in stream pipelines.

Q6. What is the difference between Function.andThen() and Function.compose()?

f.andThen(g) returns x -> g(f(x)) — f is applied FIRST, then g. f.compose(g) returns x -> f(g(x)) — g is applied FIRST, then f. Example: strlen.andThen(isEven) = first compute length, then check if even (same as isEven.compose(strlen)). Mnemonic: andThen = sequential pipeline (natural reading order); compose = mathematical composition f■g where g runs first.

Q7. What are primitive functional interface specialisations and why do they exist?

IntPredicate, LongFunction, ToIntFunction, IntUnaryOperator, etc. exist to avoid autoboxing overhead when working with primitive types. Without them, using Function boxes every int to Integer and back, creating many short-lived objects under high throughput. IntStream, LongStream, DoubleStream and their associated functional interfaces are the primitive stream counterparts. Rule: prefer primitive specialisations in performance-critical code.

Q8. How does Comparator.comparing() enable multi-level sorting?

Comparator.comparing(keyExtractor) creates a Comparator by comparing the key extracted from each element. .thenComparing(keyExtractor2) appends a secondary sort criteria used only when the first is equal. Example: Comparator.comparing(Person::getDept).thenComparing(Person::getSalary, Comparator.reverseOrder()). reversed() flips the entire comparator. nullsFirst()/nullsLast() handle null values. The chain builds a single Comparator object — immutable and reusable.

Part 2 — Stream API

Q9. Explain stream lazy evaluation with an example.

Intermediate operations (filter, map, flatMap, limit, skip, sorted, distinct, peek) are lazy — they don't execute; they build a pipeline. Only a terminal operation (collect, forEach, count, reduce, findFirst, anyMatch) triggers execution. Example: stream.filter(x>0).map(x->x*2).findFirst() stops after finding the first positive element multiplied by 2 — it does NOT process all elements. This enables short-circuit optimisation and avoids unnecessary computation.

Q10. What is the difference between map() and flatMap()?

map(): 1-to-1 transformation — each element produces exactly one output element. flatMap(): 1-to-many — each element produces a Stream, and all streams are concatenated (flattened). Example: ['hello world','java streams'].flatMap(s -> Arrays.stream(s.split(' '))) produces ['hello','world','java','streams']. flatMap avoids Stream> — use it whenever a mapping function returns a Stream or Collection.

Q11. What is the difference between findFirst() and findAny()?

findFirst() returns the first element in encounter order (respects ordered streams). findAny() may return any element — on sequential streams it usually returns the first, but on parallel streams it may return any element from any partition. findAny() can be faster on parallel streams since it doesn't require coordinating to find the first. Both return Optional and short-circuit (stop after finding one element).

Q12. What does allMatch() return on an empty stream?

true — vacuous truth. allMatch on an empty stream means 'all zero elements satisfy the predicate' which is vacuously true. Similarly: noneMatch on empty returns true. anyMatch on empty returns false. This aligns with mathematical set theory: universal quantification over an empty set is true; existential quantification is false.

Q13. What is the difference between reduce() overloads?

1) reduce(identity, BinaryOperator): always returns T (never empty — identity is the base case). Safe for parallel: identity combined with any element returns that element. 2) reduce(BinaryOperator): returns Optional — handles empty stream with empty Optional. 3) reduce(identity, BiFunction accumulator, BinaryOperator combiner): for parallel streams — accumulator combines identity with each element; combiner merges partial results from parallel threads. The combiner must be associative and compatible with the accumulator.

Q14. When should you NOT use parallel streams?

Avoid parallel streams when: 1) Small datasets — parallelism overhead exceeds benefit. 2) IO-bound operations — blocking threads ties up ForkJoinPool.commonPool() affecting other parallel streams. 3) Stateful operations — sorting, distinct require buffering; gains are minimal. 4) Ordered terminal operations — forEachOrdered negates parallel benefits. 5) Shared mutable state — ArrayList.add() in forEach causes data races. 6) Non-associative reduce — subtraction is non-associative; parallel reduce gives wrong results. Use parallel for: large (>10K), CPU-bound, stateless, unordered, embarrassingly parallel workloads.

Q15. What is a stateful intermediate operation and how does it affect stream pipelines?

Stateful operations must inspect multiple elements before producing results: sorted(), distinct(), limit(), skip(). They break pipeline fusion — the stream must buffer elements up to that point before continuing. sorted() requires all elements to be collected before any can be emitted. In parallel streams, stateful operations require coordination across threads and may eliminate parallelism benefits or require barriers. Stateless operations (filter, map) process each element independently and compose efficiently.

Q16. How do you create an infinite stream and safely terminate it?

Stream.generate(Supplier) — generates an infinite stream from a supplier (e.g., Math::random). Stream.iterate(seed, UnaryOperator) — generates infinite stream by repeatedly applying a function. Java 9+: Stream.iterate(seed, predicate, function) — finite iterate with a stop condition. Terminate with limit(n), findFirst(), anyMatch(), or other short-circuit operations. NEVER call count() or collect() on an infinite stream — it runs forever. Example: Stream.iterate(1, n->n*2).limit(10) generates 1,2,4,...,512.

Q17. What is the difference between forEach() and forEachOrdered() on parallel streams?

forEach() on a parallel stream does not guarantee encounter order — elements may be processed in any order by different threads. Faster since no ordering synchronisation is needed. forEachOrdered() guarantees encounter order even on parallel streams — but this requires synchronisation that negates most parallel benefits. Rule: use forEach for side effects where order doesn't matter (logging, inserting); use collect() (not forEachOrdered) when you need ordered results from a parallel stream.

Q18. How do Collectors.groupingBy and partitioningBy differ?

groupingBy(classifier): groups elements into a Map> where K is the result of the classifier function. Keys can be any type (String dept, Integer age range, etc.). partitioningBy(predicate): special case of groupingBy with Boolean key — always produces exactly two groups: Map> — true group and false group. partitioningBy is more efficient since it knows there are only 2 possible keys. Both accept a downstream Collector for further reduction within each group.

Part 3 — Optional & Interface Evolution

Q19. What is Optional and what problem does it solve?

Optional is a container that may or may not hold a non-null value. It makes null-ness explicit in the return type, forcing callers to handle the absent case. Solves the problem of NullPointerException from methods that may return null without documentation. Primary use: method return type. NOT a replacement for null everywhere — don't use as field type or method parameter.

Q20. What is the difference between Optional.orElse() and Optional.orElseGet()?

orElse(value): evaluates the value ALWAYS — even when Optional is present. orElseGet(supplier): evaluates the supplier ONLY when Optional is empty (lazy). If the default involves a method call (DB query, object creation), use orElseGet to avoid unnecessary work when the Optional already has a value. Rule: orElse for trivial constants; orElseGet for any computation.

Q21. What does Optional.flatMap() do differently from map()?

map(fn) wraps the result in Optional — if fn returns String, map produces Optional. flatMap(fn) expects fn to return Optional itself, and 'flattens' to avoid Optional>. Example: user.map(User::getAddress) returns Optional> if getAddress returns Optional. user.flatMap(User::getAddress) returns Optional — correct when getAddress returns Optional. Use flatMap when chaining methods that themselves return Optional.

Q22. Why should Optional not be used as a method parameter?

Using Optional as a parameter forces callers to wrap their values: method(Optional.of(value)) instead of method(value). It adds boilerplate without benefit — callers could just pass null. Better alternatives: method overloading (with and without the parameter), @Nullable annotation, or a specific absent-value constant. Optional is designed for return types to signal that absence is a common, expected outcome.

Q23. What methods did Java 9 add to Optional?

Three additions: 1) ifPresentOrElse(Consumer, Runnable): acts on value if present; runs Runnable if empty. 2) or(Supplier>): returns this if present; else returns the supplied Optional — enables fallback chain of Optional providers. 3) stream(): returns a Stream of zero or one element. Enables flatMap-based filtering: listOfOptionals.stream().flatMap(Optional::stream) removes all empty Optionals in one pass.

Q24. What is the default method diamond problem and how is it resolved?

When a class implements two interfaces both providing a default method with the same signature, the compiler cannot choose — it's a conflict. The implementing class MUST override the method. It can call a specific interface's default using InterfaceName.super.method(). Resolution rules: 1) Class implementation always wins over interface default. 2) More specific interface wins (B extends A — B's default wins). 3) Otherwise: compile error — class must override explicitly.

Advanced Questions — Senior/Principal Level

Purple [ADV] questions probe JVM internals, performance, and design.

Part 4 — Advanced Stream Internals & Performance

[ADV] Q25. How are lambda expressions compiled to bytecode?

Lambdas use invokedynamic (Java 7 instruction) + LambdaMetafactory. At the invokedynamic call site, the JVM invokes LambdaMetafactory.metafactory() which uses ASM to generate a class implementing the functional interface at runtime (first call only — then cached). The lambda body is compiled to a private static method (or instance method if it captures 'this') in the enclosing class. This is more efficient than anonymous inner classes: no separate .class file at compile time, JVM can choose optimal strategy (even reuse existing methods), and the generated class is lightweight with minimal overhead.

[ADV] Q26. What is stream pipeline fusion and when does it break?

Fusion: the JVM merges consecutive stateless operations (filter + map + filter) into a single pass through the data — no intermediate collections created. Implemented via lazy Spliterator chaining in ReferencePipeline. Fusion BREAKS at stateful operations: sorted() must collect all elements; distinct() must track seen elements; limit/skip with parallel streams need coordination. After a stateful op, a new 'stage' begins. Practical impact: filter().map() is O(n) single pass; sorted().filter() is O(n log n) + O(n).

[ADV] Q27. How does Stream.collect() work internally — what is a Collector?

Collector has three type parameters: T=input, A=accumulator container, R=result. Four components: supplier() — creates empty accumulator; accumulator() — folds one element in; combiner() — merges two partial accumulators (for parallel); finisher() — transforms A to R. Characteristics: CONCURRENT (parallel-safe accumulator), UNORDERED, IDENTITY_FINISH. Example: Collectors.toList() supplier=ArrayList::new, accumulator=List::add, combiner=addAll, finisher=identity. Custom Collector: implement Collector.of(supplier, accumulator, combiner, finisher, characteristics).

[ADV] Q28. What are Spliterator characteristics and how do they affect parallel streams?

Spliterator characteristics are bit flags: ORDERED, DISTINCT, SORTED, SIZED, SUBSIZED, IMMUTABLE, CONCURRENT, NONNULL. They inform the stream pipeline of properties it can exploit. SIZED: allows better task splitting (equal halves). ORDERED: requires maintaining encounter order (adds synchronisation cost in parallel). SORTED: skip sorting if already sorted. ArrayList Spliterator: ORDERED|SIZED|SUBSIZED — optimal parallel splitting (equal halves). HashSet Spliterator: DISTINCT — no dedup needed. LinkedList Spliterator: ORDERED but not SIZED — less efficient parallel splitting.

[ADV] Q29. Why can the combiner in reduce() differ from the accumulator for parallel streams?

In parallel, the stream splits data across threads; each thread accumulates a partial result using the accumulator. Then partial results are merged using the combiner. If identity=0, accumulator=add, and we process [1,2,3,4] in two threads: Thread 1: 0+1+2=3; Thread 2: 0+3+4=7; combiner: 3+7=10. The combiner must be associative and compatible: combiner(acc(id,e1), acc(id,e2)) == acc(acc(id,e1),e2). Subtraction violates this: (0-1)-(0-2) = -1+2 = 1 but 0-1-2 = -3. Never use subtraction in reduce.

[ADV] Q30. How does ForkJoinPool.commonPool() affect parallel stream behaviour?

parallelStream() uses ForkJoinPool.commonPool() by default. The common pool has parallelism = Runtime.availableProcessors() - 1 (leaving one for the calling thread). Problem 1: all parallel streams in the JVM share the common pool — noisy neighbours. Problem 2: blocking I/O in a parallel stream blocks a common pool thread, reducing available parallelism for other operations. Solution: wrap in a custom ForkJoinPool: customPool.submit(() -> list.parallelStream()...).get(). Java 21 virtual threads provide an alternative for I/O-bound parallelism.

[ADV] Q31. What is Collectors.teeing() (Java 12) and when is it useful?

teeing(downstream1, downstream2, merger) passes each element to TWO downstream collectors simultaneously, then merges the results with a BiFunction. Use case: compute two aggregations in a single pass without iterating twice. Example: compute min and max in one pass: Collectors.teeing(Collectors.minBy(Comp), Collectors.maxBy(Comp), (min,max)->new Range(min,max)). Without teeing you'd need two streams or a manual reduce — teeing is more elegant and efficient.

[ADV] Q32. How does Optional interact with serialisation?

Optional is NOT Serializable — intentionally. Optionals are designed for use as return types, not for persistent storage or transmission. If you put Optional in a Serializable class field, Java will throw NotSerializableException. Fix: store the unwrapped value (String or null) in the field; wrap in Optional only in the getter: public Optional getName() { return Optional.ofNullable(name); }. This is the recommended pattern — Optional stays in the API surface, not in serialised state.

[ADV] Q33. What is the internal implementation of Collectors.groupingBy()?

groupingBy(classifier) creates a Map> by: supplier = HashMap::new; accumulator = for each element e, compute key=classifier(e), then compute m.computeIfAbsent(key, k->new ArrayList()).add(e); combiner = merge two partial maps (for parallel). With a downstream collector: wraps the downstream's accumulator per key. The CONCURRENT characteristic on the downstream (e.g., ConcurrentHashMap + concurrent accumulator) allows parallel grouping without combiner — all threads write to the same map directly.

[ADV] Q34. Explain how DateTimeFormatter is thread-safe while SimpleDateFormat is not.

SimpleDateFormat stores mutable parsing state in instance fields (Calendar, NumberFormat) — concurrent calls on the same instance corrupt each other's state. DateTimeFormatter is immutable: all parsing patterns are pre-compiled into an array of DateTimePrinterParser objects at construction time. parse() and format() create local DateTimeParseContext on the stack — no shared mutable state. A single DateTimeFormatter instance can be safely shared as a static final constant, while SimpleDateFormat must be thread-local or new-per-call.

[ADV] Q35. What is the Stream.iterate() with predicate form (Java 9) and how does it improve on Java 8?

Java 8: Stream.iterate(seed, f) — infinite; must use limit() to terminate. Java 9: Stream.iterate(seed, hasNext, f) — finite; stops when hasNext(seed) is false. Example: Stream.iterate(1, n -> n <= 100, n -> n * 2) produces 1,2,4,...,64 without needing limit(). More expressive: the termination condition is co-located with the generation logic. Analogous to a for loop: for(int i=1; i<=100; i*=2). The resulting stream is ORDERED and SEQUENTIAL by default.

[ADV] Q36. How does method reference capture differ for bound vs unbound and what are the performance implications?

Bound instance ref (obj::method): captures 'obj' at the time the reference is created — like a closure over 'obj'. The generated class holds a reference to 'obj'. Unbound instance ref (Type::method): no capture — the instance is passed as the first argument each time the reference is invoked. Performance: unbound refs can be more efficiently represented (essentially a static method pointer); bound refs require the captured instance to be stored. Both are efficient after JIT inlining. Correctness concern: if 'obj' is mutable, a bound ref always uses the same object.

[ADV] Q37. What is the difference between Stream.of(null) and Stream.ofNullable(null) (Java 9)?

Stream.of(null): creates a stream with one element — null. Calling forEach prints null. But Stream.of((T)null) requires an explicit cast to avoid varargs ambiguity. Stream.ofNullable(null) (Java 9): creates an EMPTY stream if the argument is null; a stream of one element if non-null. Use case: Stream.ofNullable(possiblyNullValue).flatMap(s -> doWork(s)) — cleanly handles nullable values in a stream pipeline without null checks.

[ADV] Q38. How does Collectors.toMap() handle null values and what is the fix?

Collectors.toMap(keyMapper, valueMapper) uses HashMap.merge() internally, which throws NullPointerException if the value mapper returns null — because HashMap.merge() doesn't allow null values (it interprets null as 'remove'). Fix 1: filter out null-value entries before collecting. Fix 2: use a custom collector with computeIfAbsent instead of toMap. Fix 3: use Collectors.toMap with merge function that handles nulls explicitly. Alternatively: collect to a Map that allows null values using explicit forEach + put.

[ADV] Q39. What is the relationship between Stream, Spliterator, and Collection?

Collection.stream() calls StreamSupport.stream(spliterator(), false). The Spliterator is the bridge between a data source and the Stream API — it provides: tryAdvance() (process one element), forEachRemaining() (process remaining), trySplit() (split for parallel). Stream pipeline stages wrap each other's Spliterators lazily: filter(pred) wraps the source Spliterator with a FilteringSpliterator that calls tryAdvance and skips non-matching elements. The terminal operation drives the chain by calling forEachRemaining on the outermost Spliterator.

[ADV] Q40. How do default interface methods interact with abstract classes in Java 8?

When a class extends an abstract class AND implements an interface with a default method of the same signature, the abstract class ALWAYS wins — even if it doesn't implement the method. The abstract class declaration takes priority over interface defaults. This preserves backward compatibility: existing abstract classes are not accidentally overridden by new interface defaults added later. If the abstract class provides a concrete implementation, that is used. If the abstract class declares the method abstract, the concrete subclass must implement it.

Exercises

Part A — Coding Challenges

A1. Given List (fields: name, dept, salary), write Stream pipelines to: (a) top-3 earners per department as Map>, (b) average salary per dept as Map, (c) total salary bill for employees earning above 80k.

A2. Implement a generic memoize function: Function memoize(Function fn) using HashMap for caching. Then create a thread-safe version using ConcurrentHashMap and computeIfAbsent. Discuss why computeIfAbsent is preferred over get+putIfAbsent.

A3. Write a method using Optional chaining: findUserById(Long id) -> getAccount() -> getBalance() -> return Optional. Handle each level returning Optional. Show both the old null-check style and the Optional chain style.

A4. Parse a list of date strings in 'dd/MM/yyyy' format using Streams and DateTimeFormatter. Group them by year-month (YearMonth) and return Map>. Handle parse failures by filtering out invalid dates (not throwing).

A5. Implement a custom Collector that collects strings into a comma-separated string in REVERSE insertion order (last element first). Use Collector.of(supplier, accumulator, combiner, finisher).

Part B — Debug & Fix

B1 – Stream Reuse

```
Stream<String> stream = names.stream().filter(s -> s.startsWith("A"));
long count = stream.count();
List<String> list = stream.collect(Collectors.toList()); // IllegalStateException!
```

Fix: Streams are single-use. After a terminal operation (count), the stream is closed. Fix: create a new stream for each terminal operation, or store the source list and re-stream.

B2 – orElse Always Evaluating

```
Optional<Config> config = Optional.of(loadedConfig);
Config result = config.orElse(loadDefaultFromDatabase()); // DB call always made!
```

Fix: orElse evaluates its argument eagerly. Fix: config.orElseGet(() -> loadDefaultFromDatabase()); Now the DB call only happens when config is empty.

B3 – Parallel Stream Shared Mutable State

```
List<String> results = new ArrayList<>();
names.parallelStream().filter(s -> s.length() > 3).forEach(results::add); // RACE!
```

Fix: ArrayList is not thread-safe. Concurrent adds cause data corruption. Fix: names.parallelStream().filter(s -> s.length() > 3).collect(Collectors.toList()); Collectors handle concurrency internally.

B4 – Collectors.toMap Duplicate Key

```
Map<String,Integer> map = employees.stream()
.collect(Collectors.toMap(Employee::getDept, Employee::getSalary));
```

Fix: Multiple employees in same dept causes IllegalStateException: duplicate key. Fix: provide merge function: Collectors.toMap(Employee::getDept, Employee::getSalary, Integer::max)

Part C — MCQ / Predict the Output

MC1. What does allMatch() return on an empty stream?

A) false

B) true

C) Optional.empty()

D) throws NoSuchElementException

Answer: B — Vacuous truth: 'all elements satisfy' is true when there are no elements. anyMatch returns false on empty; noneMatch returns true on empty.

MC2. Stream.of(1,2,3).reduce(0, (a,b) -> a - b) on sequential stream gives?

A) -6

B) 6

C) -4

D) Depends on evaluation order

Answer: A — Sequential: ((0-1)-2)-3 = -6. On parallel, non-associative ops give wrong results.

MC3. opt.orElse(expensiveMethod()) when opt is non-empty:

A) expensiveMethod() is NOT called

B) expensiveMethod() IS called

C) Depends on JVM

D) Throws UnsupportedOperationException

Answer: B — orElse always evaluates its argument. Use orElseGet with a Supplier to make it lazy.

MC4. Which creates a stream of 5 odd numbers starting from 1?

A) Stream.iterate(1, n->n+2).limit(5)

B) Stream.generate(()->1).limit(5)

C) IntStream.range(1,6).filter(n->n%2==1)

D) Both A and C

Answer: D — A gives [1,3,5,7,9] using iterate. C gives [1,3,5] (range 1 to 5 exclusive, odds only 3 elements). Wait, range(1,6) gives 1..5, filtering odds: 1,3,5 = 3 elements not 5. Only A is correct.

MC5. What is the result of Predicate.not(String::isEmpty) in a stream filter?

A) Compile error — not() is not a method

B) Keeps non-empty strings

C) Keeps empty strings

D) Throws NullPointerException

Answer: B — Predicate.not(pred) (Java 11) returns the negation. not(isEmpty) = !isEmpty = keep non-empty strings.

MC6. Files.lines(path) — what must you do with the returned stream?

A) Call terminal op to close it

B) Call stream.close() manually

C) Use it in try-with-resources

D) Both B and C

Answer: D — Files.lines() returns a Stream backed by an open file handle. It implements AutoCloseable — use try-with-resources or call .close() explicitly to prevent file descriptor leak.

Quick Reference Cheat Sheet

Topic	Key Points to Remember
Lambda	(params)->expr or {block}; captured vars must be effectively final; 'this' = enclosing class
Method refs	Static::m, Type::m (unbound), obj::m (bound), Type::new (constructor)
Functional IF	Exactly 1 abstract method (SAM); default/static methods allowed; @FunctionalInterface enforces
Predicate	test(); and()/or()/negate(); Predicate.not(p) Java 11
Function	apply(); andThen(g)=f then g; compose(g)=g then f
Stream pipeline	Source -> intermediate (lazy) -> terminal (triggers execution)
Lazy evaluation	filter/map/flatMap don't run until terminal; enables short-circuit
flatMap	1-to-many; flattens Stream -> Stream; use when fn returns Stream/Collection
reduce	identity+BinaryOp->T; BinaryOp->Optional; 3-arg for parallel (needs combiner)
Parallel streams	ForkJoinPool.commonPool; good for large CPU-bound; avoid IO/stateful/small data
Collectors	toList/Set/Map; groupingBy; partitioningBy; joining; counting; downstream collectors
toMap duplicates	Provide merge function or IllegalStateException on duplicate key
Optional	Return type only; ofNullable not of(null); orElse=eager, orElseGet=lazy
Optional chains	map/flatMap/filter/ifPresent/or; stream() Java 9; avoid get() without check
Default methods	Interface evolution; diamond conflict: class wins > specific-IF > compile error
java.time	All immutable+thread-safe; LocalDate/Time/DT (no zone); ZonedDT/Instant (with zone)
DateTimeFormatter	Thread-safe (pre-compiled patterns); static constants; ofPattern for custom
Map Java8	getOrDefault, putIfAbsent, computeIfAbsent, compute, merge, replaceAll, forEach

End of Sub-Chapter 1.9 — Java 8 Features

Next: Sub-Chapter 1.10 — Java 9–21 Features